

|                  |                                                                                     |
|------------------|-------------------------------------------------------------------------------------|
| Experiment No.   | 1                                                                                   |
| Experiment Title | Environment Setup & Tooling: Working with Python data science packages <sup>1</sup> |
| Student Name     | Naren Kandasamy                                                                     |
| Register Number  | 3122247001036                                                                       |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                                        |
| Submission Date  | 02/08/2026                                                                          |

### Anaconda Activation Command:

```
source /opt/anaconda3/bin/activate anaconda-navigation
```

## 1 Objective

The purpose of this lab was to gain practical experience with the standard Python data-science libraries – Numpy, Pandas, Scipy, Scikit-learn, and Matplotlib – by applying them to real datasets. This involved practicing array handling, cleaning up messy data, building a basic machine learning pipeline, and producing an exploratory data analysis summary figure.

## 2 Problem Statement

Before any complex machine learning models can be built, a reliable data processing pipeline needs to be in place. This experiment focuses on that groundwork: bringing in datasets of different shapes and sources, dealing with missing values, encoding categorical strings into numeric form, scaling the numerical inputs, and setting up an automated way to generate exploratory visualizations.

## 3 Dataset Description

Five well-known public datasets were used for this exercise: Iris, Loan Prediction, Diabetes, Email Spam, and MNIST. Each one was looked at individually to work out what type of ML task it belongs to, an appropriate feature-selection technique, and algorithms that would be suitable for it:

| Dataset                      | Type of ML task                         | Feature Selection Technique     | Suitable ML Algorithm              |
|------------------------------|-----------------------------------------|---------------------------------|------------------------------------|
| Iris Dataset                 | Supervised (Multi-class Classification) | ANOVA (SelectKBest)             | Decision Tree, KNN                 |
| Loan Amount Prediction       | Supervised (Regression)                 | F-regression (SelectKBest)      | Linear Regression, SVM             |
| Predicting Diabetes          | Supervised (Binary Classification)      | Chi-square / Mutual Information | Logistic Regression, Random Forest |
| Classification of Email Spam | Supervised (Binary Classification)      | TF-IDF / Chi-square             | Naive Bayes, SVM                   |

| Dataset                                   | Type of ML task                         | Feature Selection Technique | Suitable ML Algorithm      |
|-------------------------------------------|-----------------------------------------|-----------------------------|----------------------------|
| Handwritten Character Recognition / MNIST | Supervised (Multi-class Classification) | PCA / Variance Threshold    | SVM, Neural Networks (CNN) |

## 4 Exploratory Data Analysis

Before any preprocessing was carried out, each dataset was examined using Pandas, Matplotlib, and Seaborn to get a sense of its structure and content. A reusable function was written to automatically pick out the most relevant numerical features and lay out exactly 12 subplots on a single page, following the required Times New Roman, 15 pt, bold axis-label format specified for the lab.

## 5 Data Preprocessing

To bring the data into a form suitable for modeling, a preprocessing pipeline was put together consisting of the following steps:

- **Handling missing values:** missing entries in numeric columns were filled in using the median, while missing categorical values were filled in using the mode.
- **Encoding:** categorical variables were converted to numeric form using simple label encoding.
- **Feature scaling:** numerical columns were scaled using `StandardScaler`.
- **Train-test split:** the datasets were split into training and testing sets, in roughly an 80%-20% ratio, using Scikit-Learn's `train_test_split`.
- **Feature selection:** `SelectKBest` was used to pick out the features that carry the most useful signal, based on statistical scoring.

## 6 Machine Learning Algorithm

Since this was an introductory experiment centered on data pipelines and exploratory analysis, no formal machine learning models were built or trained during this session. Specific algorithms will be introduced in the experiments that follow.

## 7 Experimental Setup

---

|                      |                        |
|----------------------|------------------------|
| Python Version       | 3.10+                  |
| Scikit-Learn Version | 1.0+                   |
| IDE                  | VS Code                |
| Operating System     | Linux                  |
| Hardware             | Standard CPU Workspace |

---

## 8 Hyperparameter Selection

Not applicable for this data preprocessing setup experiment.

## 9 Results

Since no models were trained during this session, quantitative performance metrics such as Accuracy, Precision, Recall, or MSE were not applicable here. A reusable metrics function was, however, implemented in `ml_utils.py` so that it is ready to use in the experiments that follow.

## 10 Result Visualization

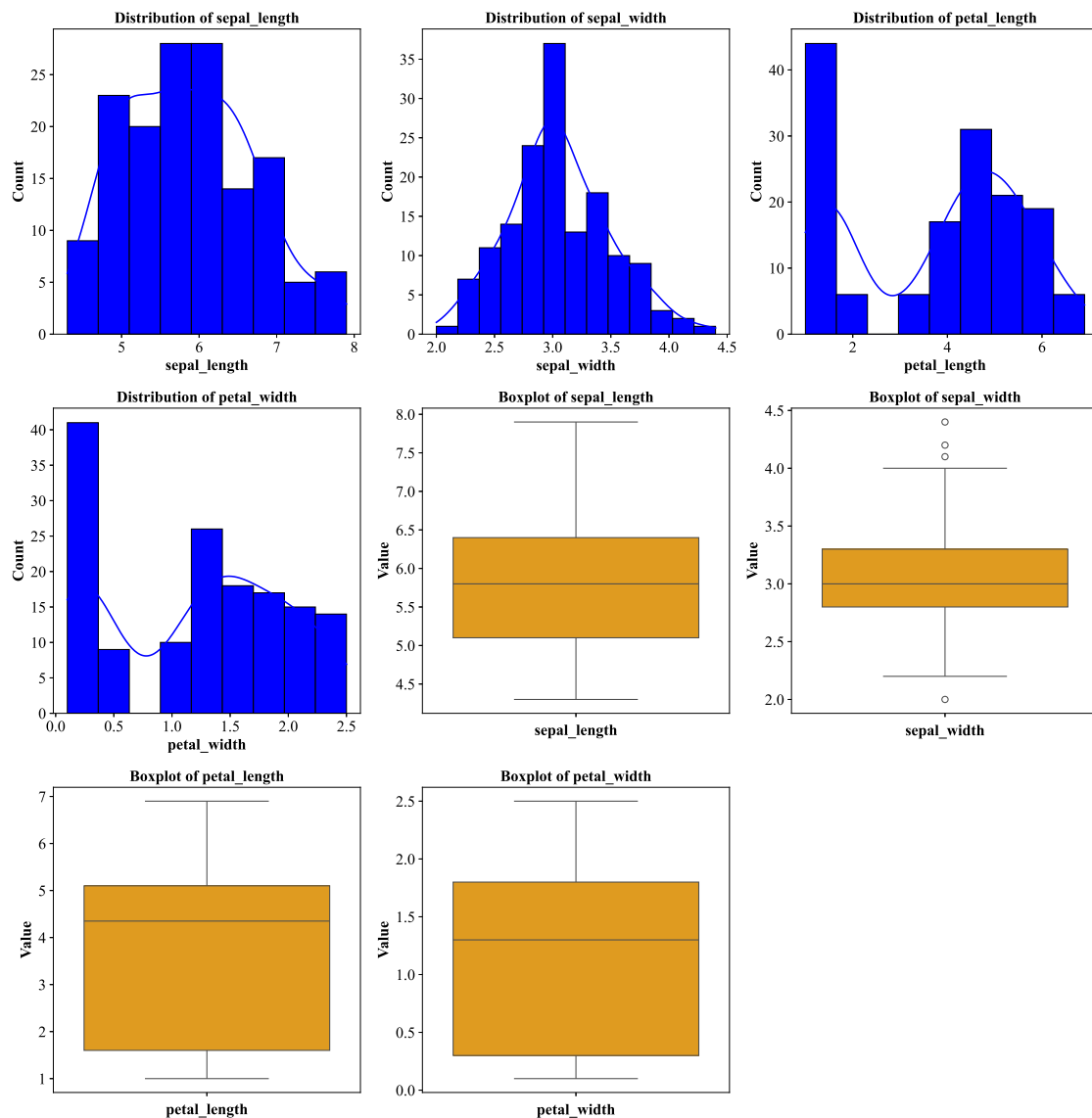


Figure 1: Consolidated EDA summary figure showcasing feature distributions and box-plots (Iris dataset).

**Inference:** Looking at the EDA plots for the Iris dataset, petal length and petal width show clearer multi-modal distributions compared to the sepal measurements, which sug-

gests these two features are likely to be more useful for distinguishing between the flower species.

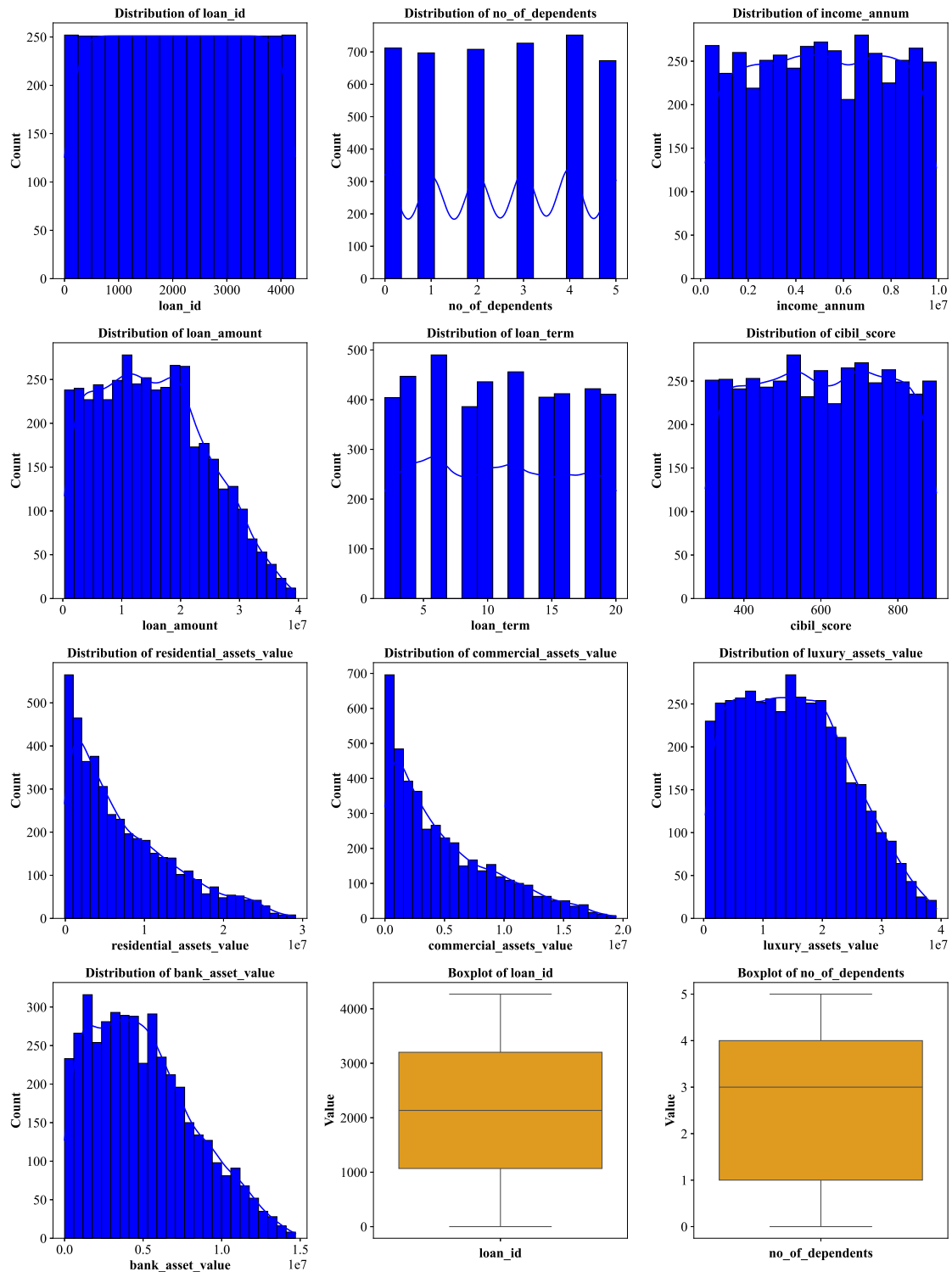


Figure 2: Consolidated EDA summary figure showcasing feature distributions and boxplots (Loan dataset).

**Inference:** The distributions show that ApplicantIncome and CoapplicantIncome are both heavily skewed, and this carries over into the target variable (LoanAmount), which

shows a similar pattern. This suggests that applying some non-linear transformation before modeling could be worth considering.

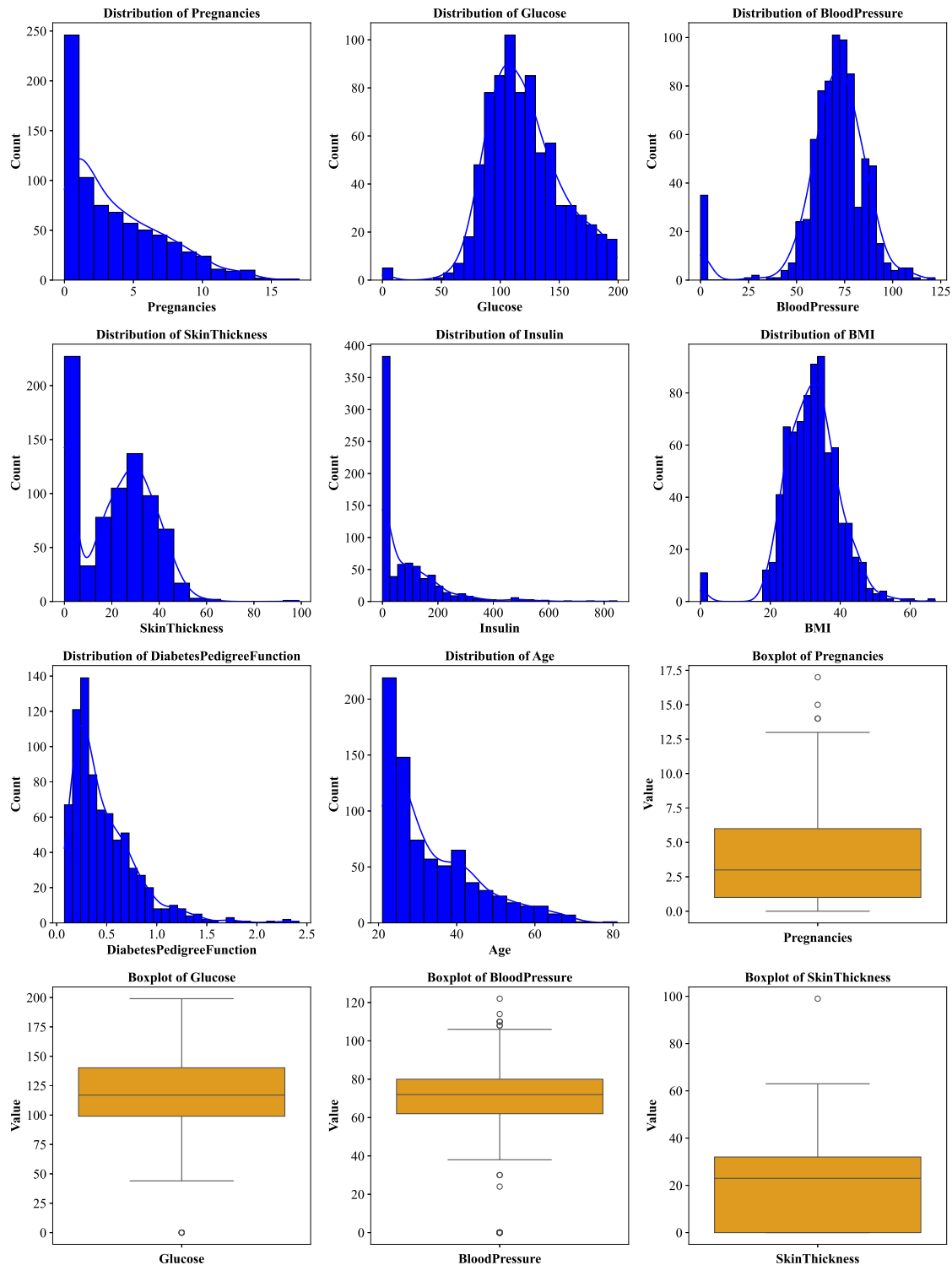


Figure 3: Consolidated EDA summary figure showcasing feature distributions and boxplots (Diabetes dataset).

**Inference:** This 12-subplot figure gives a quick overview of how the dataset's numerical features are distributed. Scanning through the histograms and boxplots together

makes it easy to spot skewness in certain variables, such as insulin, along with a number of outliers that would need to be addressed before modeling.

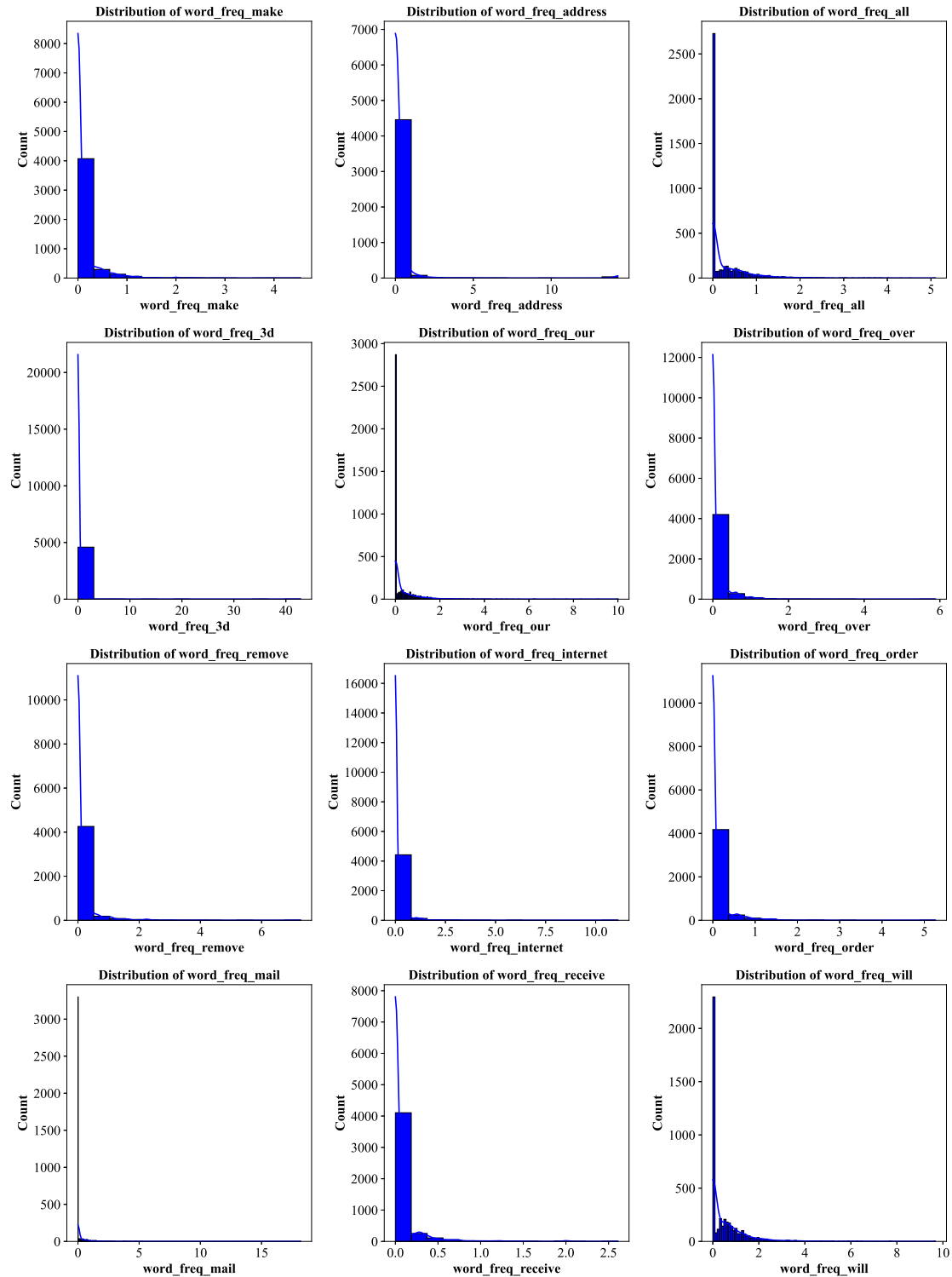


Figure 4: Consolidated EDA summary figure showcasing feature distributions and box-plots (Email Spam dataset).

**Inference:** The EDA shows long-tail distributions in several of the character-frequency features. Most of the values are concentrated near zero, but the outliers appear to carry

a fair amount of the predictive signal for identifying spam.

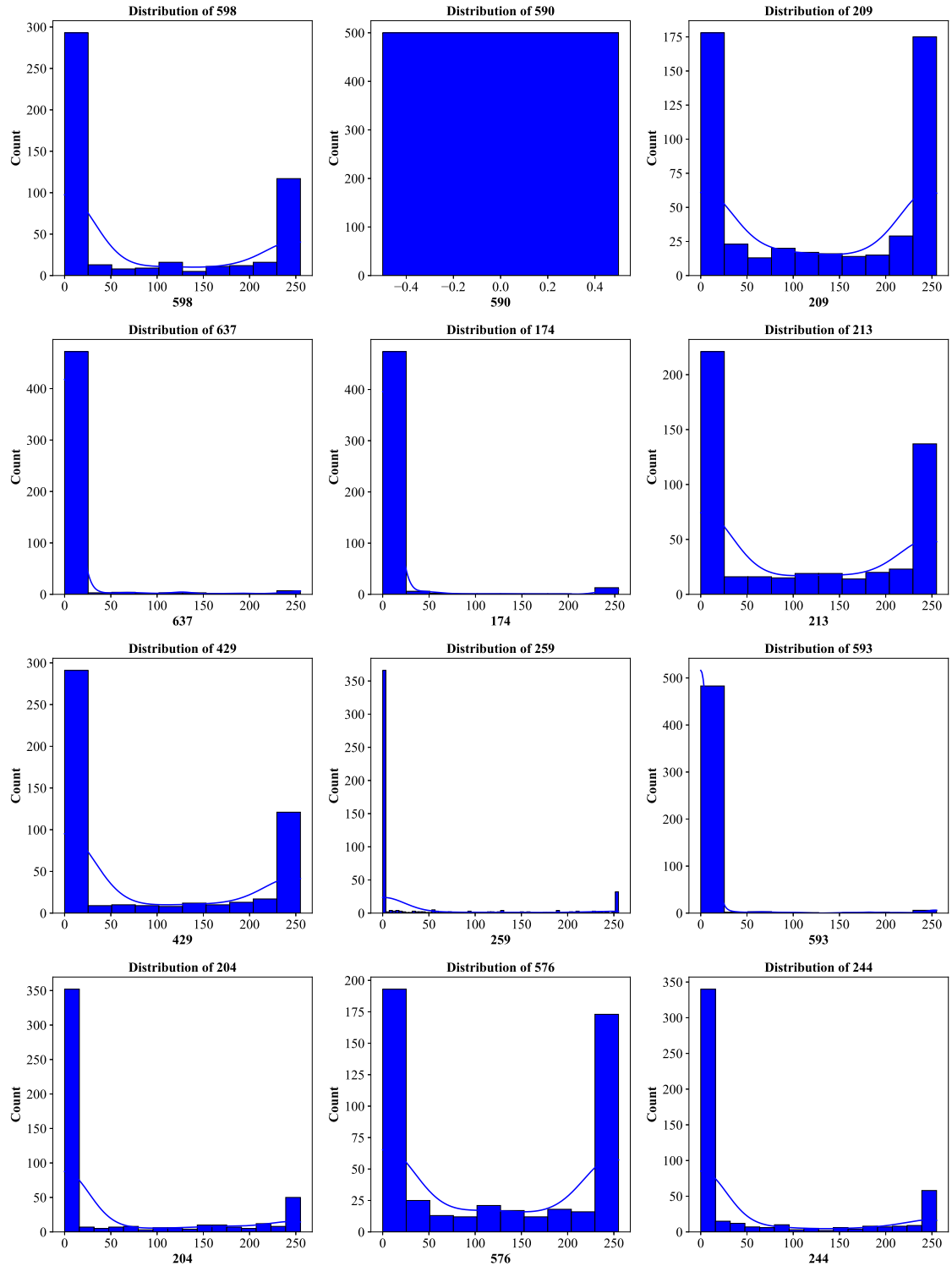


Figure 5: Consolidated EDA summary figure showcasing feature distributions and box-plots (MNIST subset dataset).

**Inference:** The pixel value distributions show that most pixels are either fully white (0) or fully black (255), with very few values in between. This sparse, near-binary structure is part of why dimensionality reduction techniques such as PCA are commonly

applied to MNIST.

## 11 Discussion

This lab made it clear why libraries such as Pandas, NumPy, and Scikit-Learn are so widely used in practice. Pandas made handling missing data and restructuring columns considerably easier, requiring very little code for tasks that would otherwise be tedious to write by hand. Scikit-Learn took care of scaling and splitting the data efficiently. Setting up the single-page, 12-subplot EDA layout also showed that Matplotlib output can be formatted programmatically to meet a fairly strict set of presentation requirements.

## 12 Conclusion

The objectives of this lab were met. Practical experience was gained with Numpy, Pandas, Scikit-Learn, and Matplotlib, and a preprocessing pipeline flexible enough to handle a range of different datasets was put together, along with a reusable plotting function that can be customized as needed. Working through each dataset individually also helped build a clearer sense of how to frame a problem correctly before moving on to model training in the experiments that follow.

## 13 References

1. McKinney, W. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 2010.
2. Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 2011.